

Assignment 1 : SDLC Analysis

DHOKHA — Software Development Life Cycle Analysis

1. Project Overview

DHOKHA is a real-time UPI fraud pattern detection system developed for a college Demo Day challenge.

The given problem statement asked the team to build a lightweight fraud scoring engine capable of flagging suspicious UPI transactions in under 200 ms using:

- behavioural patterns,
- device fingerprinting,
- and graph-based anomaly detection.

The expected output was a real-time scoring API with a demonstration interface showing fraud flags, confidence scores and explainability.

The project was developed by a **four-member team** over approximately **15 active working days**, with work distributed around a vacation-period schedule.

The development process was practical and iterative rather than a strict textbook SDLC implementation. Different parts of the system were developed in parallel and integrated progressively as they became ready.

2. Problem Analysis

The initial problem was straightforward:

Detect suspicious UPI transactions quickly and provide enough information for a user to understand why a transaction was considered risky.

During development, the team recognized that transaction-level scoring alone would not show the full fraud story. A suspicious transaction could be part of a larger network involving multiple accounts, devices or banks.

This led to an important evolution in the project:

```
Initial direction
Transaction → Fraud Score → Flag

      ↓ project evolution

Final direction
Transaction
      ↓
ML / Rule Scoring
      ↓
Relationship & Graph Analysis
      ↓
Fraud Pattern / Swarm
      ↓
Explainable Investigation Dashboard
```

The graph-based view therefore became an important part of the final product.

3. Stakeholders

For this project, the relevant stakeholders were identified from the problem and the intended demonstration system.

Stakeholder	Role in the system
Fraud Analyst / Investigator	Uses the dashboard to investigate suspicious transactions and connected accounts
Bank / Financial Institution	Source of transaction and account information
End User / UPI Participant	Generates the transactions being evaluated
Development Team	Builds and integrates the ML, backend and frontend components
Project / Demo Evaluators	Evaluate whether the system addresses the problem effectively

The system was ultimately designed around the investigator's perspective, which is why the final interface emphasized scores, explanations, live activity and graph relationships.

4. Requirements

The problem statement itself provided the main requirements. During development, these were translated into the features that the team actually implemented.

4.1 Functional Requirements

The system should:

1. Accept transaction information for scoring.
2. Calculate a fraud/risk score.
3. Identify suspicious transaction behaviour.
4. Detect connected fraud patterns using graph analysis and rules.
5. Return a fraud decision and supporting information.
6. Display transaction and fraud information through the frontend.
7. Show connected entities through graph visualisation.
8. Provide live updates through API and WebSocket communication.
9. Support demonstration scenarios for different fraud patterns.

4.2 Non-Functional Requirements

The project also had practical quality constraints:

- **Low latency:** the problem statement targeted scoring within 200 ms.
- **Real-time behaviour:** dashboard information should update as data changes.
- **Explainability:** results should show useful reasons rather than only a number.
- **Usability:** the final interface should communicate the fraud pattern clearly during the demo.

- **Maintainability:** frontend, backend, ML and deployment components were kept separated.
- **Deployment readiness:** the final application needed to run outside the local development environment.

No separate formal requirements-gathering phase was carried out because the **problem statement was already supplied by the challenge**. The team primarily interpreted it, discussed the approach, and converted it into implementation tasks.

5. SRS / Project Documentation

There was no large formal SRS document produced as a separate deliverable during the project.

Instead, the team maintained lightweight technical documentation alongside development, including:

- project planning notes,
- architecture notes,
- API descriptions,
- model and scoring details,
- graph behaviour,
- deployment instructions,
- and implementation references.

This was appropriate for the scale and time constraints of the Demo Day project.

The repository contains dedicated planning and architecture documents documenting the intended frontend, backend, graph, ML and integration approach.

6. Development Model

Selected Model: Agile — Iterative and Incremental

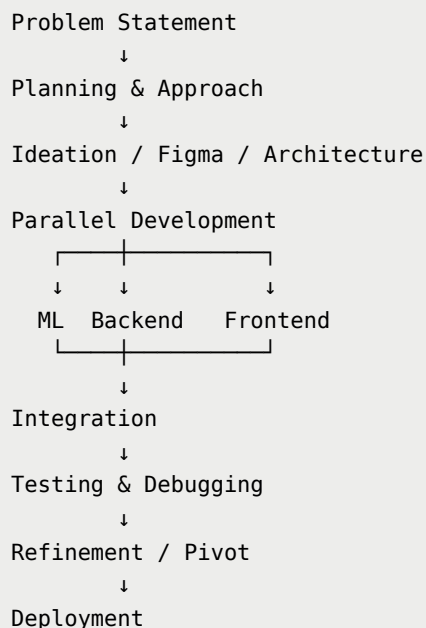
The development process is best described as an **Agile-style iterative and incremental approach**.

It was not a formal Scrum implementation: the team did not follow a complete set of Scrum ceremonies or maintain a formal sprint process.

However, the project had the important characteristics of iterative development:

- work was split into independent areas,
- multiple components were developed in parallel,
- progress was integrated repeatedly,
- changes were made after seeing the working system,
- mock data was replaced with live services,
- UI and architecture were refined late in development,
- testing and debugging happened during integration,
- and the product was progressively brought toward a final deployable state.

Actual development cycle



7. Why Agile Fit This Project

Agile was suitable because the team was working under a short deadline and different parts of the application had to progress simultaneously.

For example:

- the ML developer could work on the model without waiting for the frontend,
- the frontend could be built using mock data before the backend was complete,
- the backend could expose APIs while the UI was still evolving,
- and the final integration could replace the temporary mock layer with real API and WebSocket communication.

The approach also supported late changes.

A good example was the evolution from a basic fraud-scoring concept toward a more visual graph-driven investigation experience. The main dashboard, graph explorer and other pages were refined once the real backend behaviour was available.

This made the development process **iterative rather than linear**.

8. Development Timeline

The project had gaps in calendar dates because it was developed during vacation. Therefore, it is more accurate to describe the schedule as approximately **15 active working days**.

Active days	Focus	Outcome
Days 1–2	Planning and problem discussion	Overall approach, responsibilities, repository setup and initial direction
Days 3–5	Ideation, Figma and mockups	Landing direction, page structure, core UI ideas and architecture exploration
Days 6–8	UI/UX + ML/backend coordination	Initial frontend foundation, ML progress and backend integration path

Active days	Focus	Outcome
Days 9–11	Core UI flow + backend APIs	Main user flow and backend API layer becoming functional
Days 12–14	Integration + testing	Frontend connected to live APIs/WebSockets, graph/scoring integration and removal of mock data
Day 15	Fixes, refinement and deployment	UX changes, debugging, final fixes, deployment and Demo Day preparation

9. Team Structure and Responsibilities

The team had four members.

ML Developer

Focused mainly on:

- fraud model development,
- model scoring,
- related feature work,
- and coordination with the backend developer.

Backend Developer

Focused mainly on:

- FastAPI services,
- scoring APIs,
- graph processing,
- persistence,
- live backend services,
- and backend deployment.

Frontend Developer

Focused mainly on:

- frontend implementation,
- page development,
- routing,
- user flow,
- and UI-side integration.

Integration / Frontend Lead

My role was cross-functional.

I:

- worked closely with the frontend developer,
- oversaw the frontend direction,
- worked on the major revamped pages myself,
- maintained the main repository,
- coordinated the ML and backend work where the components depended on each other,
- handled parts of the frontend/backend integration,
- fixed model-scoring and graph-related metrics where needed,
- handled debugging,
- made architecture and integration decisions,
- managed the final integration,
- and handled deployment.

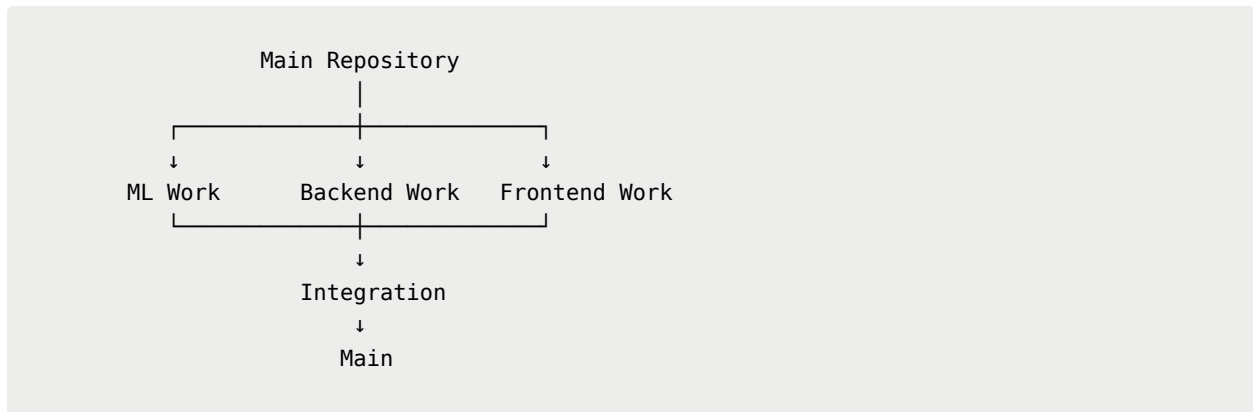
The final Demo Day presentation was a **team effort**, with all members involved in the completed product.

10. GitHub and Version Control

The main project repository was maintained centrally.

Other team members worked from their own forks and were added as collaborators, allowing them to develop independently while the main repository remained the integration point.

The workflow can be represented as:



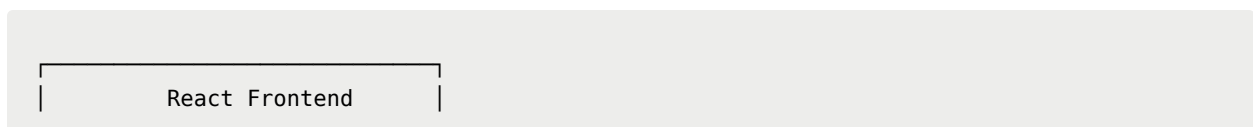
Git history shows the project evolving through multiple stages, including:

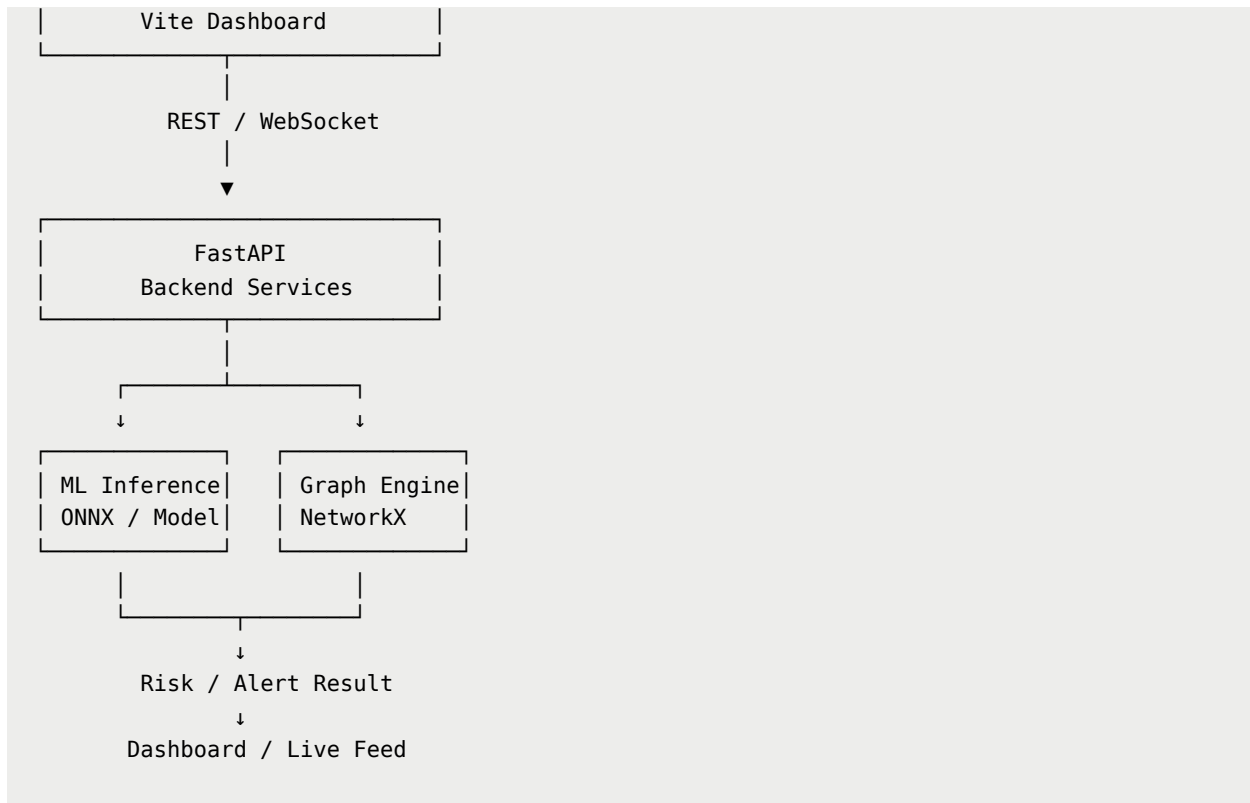
- initial planning and page drafts,
- development of the working demo,
- live backend integration,
- graph and scoring improvements,
- dashboard and graph explorer revisions,
- timestamp and compatibility fixes,
- and final integration-related changes.

This demonstrates the use of version control as part of the actual development process.

11. Architecture

The final system was built as a frontend + backend + ML + graph-based application.





The backend documentation records FastAPI, ONNX Runtime, NetworkX, SQLite/SQLAlchemy and the supporting scoring pipeline as major components of the implementation.

12. UML / Design Artifacts

For the project analysis, the most useful design views are:

Use Case View

Primary actor: Fraud Analyst

Main interactions:

- View transactions
- Score a transaction
- Inspect fraud result
- Explore connected accounts

- View fraud explanations
- Monitor live activity

Sequence View

```
graph TD; Transaction --> Frontend; Frontend --> FastAPI; FastAPI --> Feature[Feature / Data Processing]; Feature --> ML[ML + Rule + Graph Analysis]; ML --> Risk[Risk Decision]; Risk --> API[API / WebSocket]; API --> Dashboard;
```

Architecture View

The layered architecture above represents how the main software components interact.

These diagrams explain the project architecture without claiming that a formal UML design phase was carried out independently.

13. Fraud Patterns Implemented

One of the major developments in the project was moving beyond isolated transaction scoring and visualising connected fraud patterns.

Type A — Identity Fan-Out

One identity is associated with multiple accounts across banks, creating a connected pattern.

Type B — Mule Fan-In

Multiple accounts send money toward a common collector or mule account.

Type C — Layering Chain / Ring

Money moves through several accounts in sequence and can form a cycle.

Type D — Device / IP Cluster

Multiple apparently unrelated accounts are associated with the same device or infrastructure.

These patterns are naturally represented using graph relationships, which became a major part of the final dashboard.

14. Integration Process

Integration was one of the most important parts of the project because the system was developed by different people in parallel.

The early frontend used mock data so that UI work could continue while the backend was still being implemented.

Once the backend APIs and WebSocket layer were ready, the final integration phase involved:

1. connecting frontend pages to real APIs,
2. replacing mock data with live responses,
3. connecting live WebSocket events,
4. aligning backend response structures with frontend expectations,
5. fixing scoring and graph-related metrics,
6. correcting timestamp behaviour,
7. validating the integrated user flow,
8. refining the dashboard presentation.

This is where the separate workstreams became one working application.

15. Testing and Bug Fixing

Testing was performed alongside integration rather than being treated as a completely separate final phase.

The repository contains tests around API behaviour, graph functionality, scoring/rules and model integration.

Examples of practical validation included:

Test / Validation	Expected result
Submit a normal transaction	Valid transaction result returned
Submit a suspicious transaction	Elevated risk result returned
Trigger a known swarm pattern	Expected swarm pattern identified
Open a graph view	Correct connected entities displayed
Generate live backend event	Dashboard updates
Send invalid API input	Appropriate validation response
Run frontend build	Successful production build
Run backend tests	Test suite passes

The Git history also shows concrete debugging and fixes involving:

- backend/frontend scorer compatibility,
- scoring metrics,
- graph integration,
- timestamp conversion,
- live dashboard/API connectivity,
- and final UI consistency.

These were practical issues encountered while integrating the system.

16. Software Risks

The major risks relevant to DHOKHA were:

Risk	Possible impact	Mitigation used / considered
ML scoring behaves unexpectedly	Incorrect fraud classification	Scoring tests and metric validation
Backend/frontend mismatch	Broken user flow	Integration testing and API compatibility fixes
Graph logic errors	Incorrect fraud relationships	Graph-specific testing and debugging
Live WebSocket failure	Dashboard becomes stale	Fallback to API-based data and connection checks
Integration delays	Final demo not ready	Parallel development and early mock-based UI work
Late UX changes	Reduced time for testing	Prioritised core integration before final polish
Deployment problems	Demo unavailable	Deployment testing and final fixes

17. Project Planning

A simple Work Breakdown Structure for the project is:

1. Problem Understanding
2. Planning and Architecture
3. UI/UX Exploration
4. ML Development
5. Backend Development
6. Frontend Development
7. Integration
8. Testing and Debugging
9. Deployment
10. Demo Preparation

Gantt-style phase plan

Phase	Active time
Planning	Days 1–2
Ideation + Figma	Days 3–5
ML + UI + Backend foundation	Days 6–8
Core UI + APIs	Days 9–11

Integration + Testing
Fixes + Deployment

Days 12–14
Day 15

Agile-style artifacts used in practice

The team did not maintain a formal Scrum board, but the development process naturally produced lightweight Agile-style artifacts:

- work was split into tasks,
- each member owned a technical area,
- components were developed incrementally,
- integration exposed new tasks,
- changes were prioritised toward the final demo,
- and the working product was repeatedly refined.

18. SDLC Comparison

Although the project used an iterative Agile-style approach, it can be compared with Waterfall and Spiral.

Factor	Agile / Iterative	Waterfall	Spiral
Requirement changes	Easy to accommodate	Difficult after earlier phases	Easy
Parallel work	Well suited	Limited	Possible
Feedback	Frequent	Usually later	Frequent
Risk focus	Continuous practical feedback	Mostly phase-based	Strong risk analysis
Delivery	Incremental	Mostly sequential	Incremental
Suitability for DHOKHA	High	Low	Medium–High

Why not Waterfall?

The project changed during development, especially around the graph visualisation and final UX. A strictly sequential model would have made these changes harder to absorb.

Why not Spiral?

Spiral is strongly centered on formal risk analysis and repeated risk-driven cycles. That was not how this project was organised.

Why Agile / Iterative?

The actual process involved:

| parallel development → integration → testing → refinement → deployment

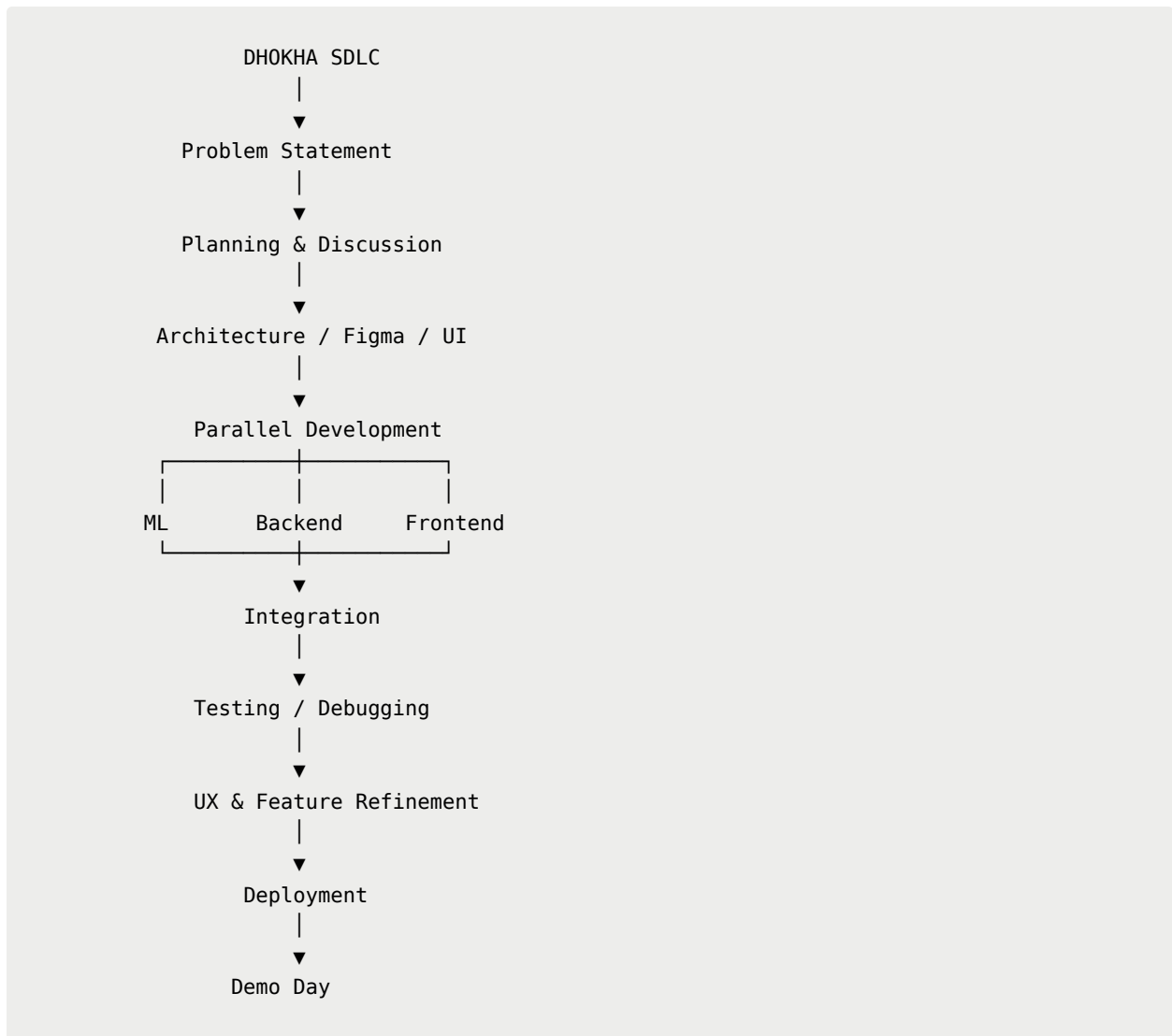
which matches an iterative and incremental development style much more closely.

19. Software Engineering Tools Used

Tool	Use in the project
GitHub	Central repository, collaboration and version history
Git	Version control and integration
Figma	UI exploration, mockups and iterative design
React + Vite	Frontend implementation
FastAPI	Backend API and WebSocket services
NetworkX	Graph-based fraud analysis
ONNX Runtime	Model inference
SQLite / SQLAlchemy	Persistence
Docker	Backend containerisation
AWS	Deployment

20. Final Development Lifecycle

Putting the actual project process together:



The important point is that this was **not a one-direction waterfall flow**. After integration and testing, the team returned to the UI, scoring logic and graph behaviour and improved them before the final deployment.

That repeated loop is the strongest evidence for the iterative nature of the project.

21. Final Outcome

Within approximately 15 active working days, the team moved from the supplied problem statement to a working end-to-end demonstration system.

The final product combined:

- ML-based transaction scoring,
- graph-based fraud pattern detection,
- rule-based analysis,
- interactive fraud investigation views,
- live API communication,
- WebSocket updates,
- explainability,
- and deployment.

More importantly, the project demonstrated the practical software-development lifecycle:

Plan → Build in Parallel → Integrate → Test → Refine → Deploy

For a small, time-constrained college project, this was a more accurate representation of the team's actual process than claiming a formal heavyweight SDLC.

22. Key Takeaway

The main lesson from DHOKHA was that building a multi-part software system is as much about **integration and coordination** as it is about writing individual components.

The ML model, backend, graph engine and frontend were developed by different people, but the final product only became usable once those parts were connected, tested and refined together.